Vibe Coding

当自然语言成为编程接口

Advanced Software Engineering · 2026

一个值得先回答的问题

> 如果实现一个功能只要 20 分钟，软件工程师的时间应该花在哪里？

定义问题	设计边界	验证行为	维护系统

这份幻灯片的迭代过程，本身也是一个小例子。

定义：Vibe Coding 是什么？

用自然语言驱动 AI 生成、修改、运行和调试代码；人类负责目标、上下文、反馈与验收。

自然语言优先

结果导向

持续反馈

人类保留判断

这个词的原始含义，其实有点激进

> “There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.”
>
> — Andrej Karpathy, 2025

极端版

先让感觉带路，跑通就好；适合一次性原型和探索。

工程版

让 AI 写更多代码，但用规格、测试和审查管理风险。

Vibe Coding 不是什么

✘ 不是“不用理解计算机系统”

✘ 不是“跳过测试和代码审查”

✘ 不是“模型永远比人更懂业务”

✘ 不是“自然语言会取代所有编程语言”

> Vibe coding 是一种协作方式，而不是质量保证。

它不是一种工具，而是一条协作光谱

模式	人的核心动作	AI 的核心动作
自动补全	写主体代码	预测下一段
对话式协助	描述、修改、审查	解释、生成片段
Vibe coding	定目标、给反馈、验收	端到端实现与迭代
Agentic engineering	设规格、管风险、担责任	规划、执行、验证

越往右，人的投入越从“敲代码”转向“定义与判断”。

为什么是现在?

能力

模型开始具备规划、工具使用和长链路推理能力。

上下文

能理解整个仓库，而不只是当前文件。

反馈

能执行命令、读取错误、修改并再次验证。

LLM + Tools + Feedback Loop = Coding Agent

什么变便宜了，什么反而更贵了？

变便宜

- 第一版实现
- 样板代码
- 测试草稿
- 方案探索

变昂贵

- 问题定义
- 领域判断
- 验证与安全
- 长期维护

瓶颈从“把代码打出来”转向“判断什么代码值得留下”。

软件抽象又向上走了一层

机器码 → 汇编 → 高级语言 → 框架 / API →

自然语言 + Agent

收益

人可以在更高的意图层工作，减少机械转换。

代价

表达更模糊，风险也更难直接看见。

抽象越高级，验收越重要。

传统循环 vs AI 密集循环

传统

写代码

→ 编译

→ 调试

→ 提交

AI 密集

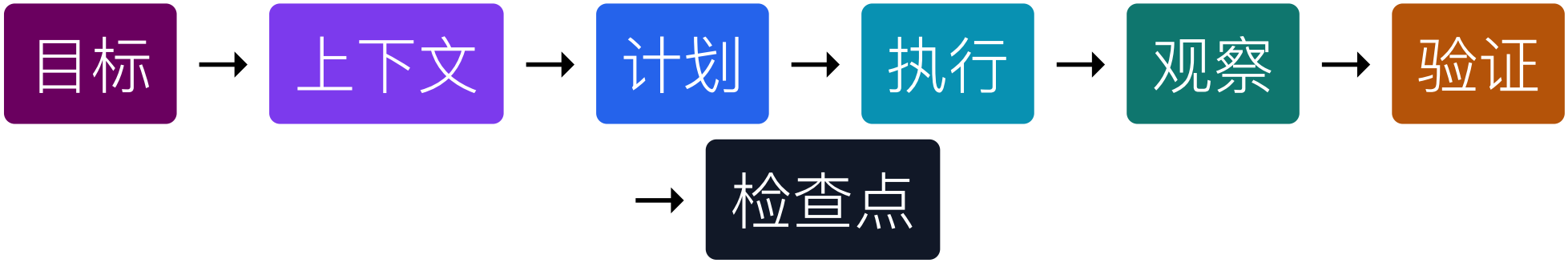
说清意图 → 提供上下文

→ 计划 → 执行

→ 观察 → 验证 → 检查点

工作单元从“多少行代码”变成“一个可验证的增量”。

核心循环：小步、可运行、可回退



1. 每次只推进一个**可观察的结果**
2. 每轮都留下**可运行的证据**
3. 每个关键步骤都保持**可回退**

> 不要一次许愿一个系统；一次证明一个增量。

Context engineering：上下文也是程序的一部分

应该提供

- 目标与非目标
- 相关文件与仓库地图
- 领域规则与样例
- 约束与验收标准
- 真实错误与日志

不要堆砌

- 整仓库无差别倾倒
- 过期文档
- 密钥与隐私数据
- 互相矛盾的指令

更好的上下文，比更长的 Prompt 更重要。

任务拆分：把“大愿望”变成“小证据”

反模式	更好的做法
“把整个系统做出来”	先做一条端到端纵切
同时重构和加功能	分开提交、分开验证
“优化一下”	给出可观察的指标
直接猜 Bug	先稳定复现问题

每轮只改变一个可验证的结果。

Prompt 是新的“需求接口”

目标：为现有项目增加 `CSV` 导出功能
上下文：技术栈、相关文件、数据样例
约束：不改变现有 `API`；不新增依赖
验收：测试通过；字段和排序正确
协作：先给计划和风险，再修改代码

具体 > 抽象

样例 > 形容词

验收 > 期待

先要计划，再要代码

1. 会改哪些文件？为什么？
2. 有哪些关键假设？
3. 最大风险是什么？
4. 如何测试？
5. 最小安全改动是什么？

> 让 AI 提出方案，让人批准范围。

在真实代码库里工作

- 先读，再写
- 复用现有模式
- 保持 diff 小
- 不随意升级依赖
- 不确定就提问
- 运行现有检查

> 好的 Agent 像一个谨慎的新同事，而不是一个自信的陌生人。

Agent 执行循环：自动化不等于失控



- 把**真实错误**和上下文一起反馈
- 要求先解释假设，再继续修改
- 连续失败时停下来，而不是反复试错
- 最后总结改动、测试和剩余风险

> 没有可观测性的自主，只是赌博。

验证阶梯：自主性越强，证据要越硬

L5 生产可观测性：日志、指标、告警

L4 审查与安全：静态检查、依赖、威胁面

L3 人工验收：真实场景与业务规则

L2 集成 / E2E：组件之间真的能协作

L1 单元测试：行为与边界条件

L0 编译 / 类型 / lint：最基本的正确性

“能运行”只是起点，不是验收。

如何审查 AI 生成的代码?

- 边界条件与失败模式
- 错误处理与重试策略
- 安全、密钥与权限
- 依赖变化与供应链
- 隐藏耦合与复杂度
- 测试是否真的验证行为

> 生成代码没有更低的审查标准。尤其要检查它“没做什么”。

检查点与可逆性

推荐

- 每个任务一个分支
- 小 commit
- 功能开关
- 备份与回滚方案

高风险

- 不可逆数据操作
- 大范围自动替换
- 无审查依赖升级
- 没有基线就重构

跑得越快，干净的“撤销”就越值钱。

什么场景适合 Vibe Coding?

场景	可逆	不可逆
熟悉领域	✅ 优先尝试	⚠️ 强化审查与回滚
未知领域	🧪 适合快速探索	⛔ 先研究，再让人决策

原型、脚本、测试、文档、局部重构通常是低风险入口；认证、支付、迁移、合规与安全关键路径需要更高门槛。

案例：一个 CSV 对账工具

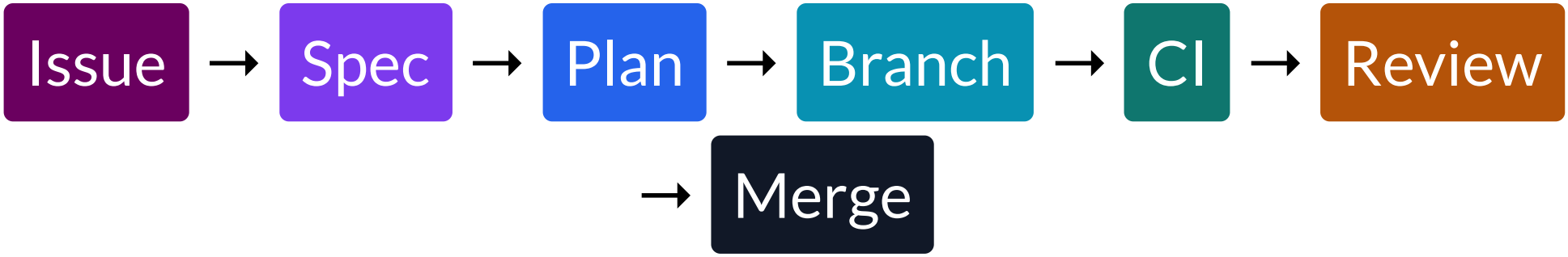
```
目标：实现 CSV 对账 CLI
约束：不覆盖输入；金额使用 Decimal
验收：3 个样例通过；差异写入 report.csv
协作：先读项目，再给计划和风险
```

AI：生成实现、测试和文档

人：确认业务规则与异常口径

AI 很快做出了“能跑的版本”；真正的价值在于发现：空值、时区和舍入规则必须先定义。

团队工作流：人仍然是变更的所有者



- 工程师：定义验收、架构边界和风险
- Agent：实现、运行、迭代、汇报
- CI：提供可重复的证据
- Reviewer：对可维护性与后果负责

> 可以委托执行，不能委托责任。

如何衡量它是否真的有用?

容易误导的指标	更有意义的指标
生成代码行数	交付周期
Prompt 数量	变更失败率
“AI 写了多少”	逃逸缺陷与返工
Token 消耗	审查延迟与开发者体验

速度如果没有质量，只是在更快地积累技术债。

风险不会消失，只会转移

幻觉 API / 逻辑错误 → 可执行测试、类型检查

提示注入 / 越权操作 → 明确信任边界与最小权限

依赖投毒 / 供应链 → 锁定版本、审查新增依赖

数据泄露 → 脱敏、私有边界、密钥管理

维护债 / 隐藏复杂度 → 小 diff、文档、可读性审查

过度信任 → 人工签字与最终责任

> Vibe coding 不是“不看代码”，而是“把验证前移”。

六个常见反模式

① 一口吃成胖子

② 只看 Demo，不看退出条件

③ 让 Agent 无限自动修复

④ 不加控制地产生大 diff

⑤ 没有版本控制就重构

⑥ 把“生成得快”当成“设计得好”

真正稀缺的不是代码，而是清晰的目标、可靠的判断和可维护的系统。

从 Vibe Coding 到 Vibe Engineering

Vibe coding	Vibe engineering
快、松、偏探索	快，但可追责
感觉对就继续	规格、测试、审查
适合一次性原型	适合长期维护的系统
依赖即时反馈	依赖工程证据

业界也越来越用 **Agentic Engineering** 描述这种专业版本：AI 大量执行，人负责规格、验证与后果。

工程师应该继续加强什么？

定义问题	设计边界	读懂代码
验证行为	管理风险	对结果负责

模型越强，越值钱的是那些能够判断输出质量的能力。

让 AI 加速探索，而不是替代判断

- ✅ Vibe coding 改变的是杠杆，不是责任
- ✅ 上下文与验收，是新的编程接口
- ✅ 自主性越强，验证越要硬
- ✅ 最好的团队用 AI 提高上限，而不是降低标准

少写代码，不等于少做工程。